



**UNIVERSIDAD
SERGIO ARBOLEDA**

Taller 1 corte 2

Lenguajes de programación

**Yslen Natalia Moreno Gutierrez
Santiago Patiño Sarmiento**

*Universidad Sergio Arboleda
Escuela de ciencias exactas e ingeniería
Bogotá D.C., 09 de septiembre de 2026*

1. Introducción

Se implementaron en ANTLR 4.13.2, con Python como lenguaje objetivo, las tres gramáticas propuestas en clase, y se analizaron los árboles que cada una produce. El trabajo cubre tres puntos: (1) la gramática clásica de expresiones de la diapositiva 11, con pruebas de evaluación; (2) las distintas formas del árbol de análisis para la gramática de la diapositiva 12; y (3) la gramática de la diapositiva 15, para comprobar si es ambigua.

La evaluación numérica se resolvió en todos los casos con el patrón **Visitor**, generado por ANTLR con la opción `-visitor`.

Entorno de trabajo

Sobre Ubuntu se configuró el `CLASSPATH` y los alias de ANTLR, y se instaló el *runtime* de Python en la misma versión del generador (4.13.2); si las versiones no coinciden, el *parser* generado falla al importarse.

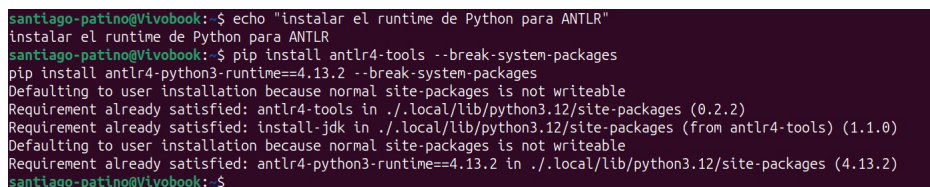
```
echo 'export CLASSPATH=".:$HOME/antlr-4.13.2-complete.jar:$CLASSPATH"' >> ~/.bashrc
echo 'alias antlr4="java -jar $HOME/antlr-4.13.2-complete.jar"' >> ~/.bashrc
echo 'alias grun="java org.antlr.v4.gui.TestRig"' >> ~/.bashrc
pip install antlr4-tools antlr4-python3-runtime==4.13.2 --break-system-packages
```

Listing 1: Alias, CLASSPATH e instalación del runtime



```
santiago-patino@Vivobook:~$ echo 'export CLASSPATH=".:$HOME/antlr-4.13.2-complete.jar:$CLASSPATH"' >> ~/.bashrc
santiago-patino@Vivobook:~$ echo 'alias antlr4="java -jar $HOME/antlr-4.13.2-complete.jar"' >> ~/.bashrc
santiago-patino@Vivobook:~$ echo 'alias grun="java org.antlr.v4.gui.TestRig"' >> ~/.bashrc
santiago-patino@Vivobook:~$ source ~/.bashrc
santiago-patino@Vivobook:~$ ls
antlr-4.13.2-complete.jar  Escritorio  Música  proyectos  Videos
Descargas                 Imágenes   Plantillas  Público
Documentos                lib        proyecto-dsl-datos  snap
santiago-patino@Vivobook:~$
```

Figura 1: Configurar Alias



```
santiago-patino@Vivobook:~$ echo "instalar el runtime de Python para ANTLR"
instalar el runtime de Python para ANTLR
santiago-patino@Vivobook:~$ pip install antlr4-tools --break-system-packages
pip install antlr4-python3-runtime==4.13.2 --break-system-packages
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: antlr4-tools in ./local/lib/python3.12/site-packages (0.2.2)
Requirement already satisfied: install-jdk in ./local/lib/python3.12/site-packages (from antlr4-tools) (1.1.0)
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: antlr4-python3-runtime==4.13.2 in ./local/lib/python3.12/site-packages (4.13.2)
santiago-patino@Vivobook:~$
```

Figura 2: Instalar el runtime de Python para ANTLR

2. Punto 1. Gramática de expresiones (diapositiva 11)

2.1. Gramática

La gramática de partida es $E \rightarrow E + T \mid T, T \rightarrow T * F \mid F, F \rightarrow id \mid num \mid (E)$. Su valor no está solo en reconocer las cadenas válidas, sino en que la **precedencia y la asociatividad quedan codificadas en la forma de las reglas**: como T está subordinado a E , todo producto queda por debajo de una suma en el árbol; y como la recursión es por la izquierda, los operadores del mismo nivel se agrupan de izquierda a derecha.

A la versión ANTLR se le añadieron la resta (necesaria para $2 + 3 - 4$) y la multiplicación implícita (para $3(4 - 5)$). Las etiquetas (**# SumaExpr**, etc.) son las que hacen que ANTLR genere un método **visit** por alternativa.

ANTLR4 admite recursión izquierda directa y la reescribe internamente, de modo que $e : e '+'$ t se puede escribir tal cual, sin las transformaciones que exigiría un generador LL clásico.

```

GNU nano 7.2                               Expr.g4
grammar Expr;

prog
    : e EOF
    ;

e
    : e '+' t      # SumaExpr
    | e '-' t      # RestaExpr
    | t            # TerminoComoExpr
    ;

t
    : t f          # MultiplicacionImplicita
    | t '*' f      # MultiplicacionExplicita
    | f            # FactorComoTermino
    ;

f
    : ID           # FactorID
    | NUM          # FactorNum
    | '(' e ')'    # FactorParentesis
    ;

ID : [a-zA-Z_][a-zA-Z_0-9]* ;
NUM : [0-9]+ ('.' [0-9]+)? ;
WS : [ \t\r\n]+ -> skip ;

```

Figura 3: Gramatica

```

santiago-patino@lvobook:~/Escritorio/analisis-sintactico/puntos$ echo "Generar el parser/lexer en Python"
Generar el parser/lexer en Python
santiago-patino@lvobook:~/Escritorio/analisis-sintactico/puntos$ antlr4 -Dlanguage=Python3 -visitor Expr.g4
santiago-patino@lvobook:~/Escritorio/analisis-sintactico/puntos$ ls
Expr.g4  ExprLexer.interp  ExprLexer.tokens  ExprParser.py  ExprVisitor.py
Expr.interp  ExprLexer.py  ExprListener.py  Expr.tokens
santiago-patino@lvobook:~/Escritorio/analisis-sintactico/puntos$

```

Figura 4: generar lexer/parser en Python

La primera versión de **main.py** solo construye el árbol y lo imprime con **toStringTree**; la segunda añade la evaluación.

```

GNU nano 7.2 main.py
# main.py

import sys
from antlr4 import InputStream, FileStream, CommonTokenStream
from ExprLexer import ExprLexer
from ExprParser import ExprParser

def probar(entrada_stream, etiqueta):
    lexer = ExprLexer(entrada_stream)
    tokens = CommonTokenStream(lexer)
    parser = ExprParser(tokens)
    arbol = parser.prog()
    print(f'{etiqueta}' -> {arbol.toStringTree(recog=parser)})

def main():
    if len(sys.argv) > 1:
        # Caso 1: se pasó un archivo como argumento
        # Uso: python3 main.py entrada.txt
        ruta = sys.argv[1]
        entrada = FileStream(ruta, encoding='utf-8')
        probar(entrada, f"archivo:{ruta}")
    else:
        # Caso 2: no hay argumento -> pedir texto por consola
        texto = input("Ingrese la expresión a analizar: ")
        entrada = InputStream(texto)
        probar(entrada, texto)

if __name__ == '__main__':
    main()

```

Figura 5: Main

```

santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ echo "Pruebas de la gramatica (solo mostrar arbol)"
Pruebas de la gramatica (solo mostrar arbol)
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ cat entrada.txt
2 + 3 * 4
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ python3 main.py entrada.txt
'archivo:entrada.txt' -> (prog (e (e (t (f 2))) + (t (t (f 3)) * (f 4))) <EOF>)
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ cat entrada1.txt
2+3-4
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ python3 main.py entrada1.txt
'archivo:entrada1.txt' -> (prog (e (e (e (t (f 2))) + (t (f 3))) - (t (f 4))) <EOF>)
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ python3 main.py entrada2.txt
'archivo:entrada2.txt' -> (prog (e (e (t (f 2))) + (t (t (f 3)) * (f ( (e (e (t (f 4))) - (t (f 5))) )))) <EOF>)
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$ cat entrada2.txt
2+3*(4-5)
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto1$

```

Figura 6: Prueba de la gramatica

El evaluador hereda de la clase generada `ExprVisitor` y define un método por alternativa. La evaluación es un recorrido en profundidad: cada método pide el valor de sus hijos y los combina.

Los identificadores se reconocen pero no se pueden evaluar: no hay tabla de símbolos, así que el evaluador lanza un error explícito en vez de fallar en silencio.

```
GNU nano 7.2
from ExprVisitor import ExprVisitor

class ExprEvalVisitor(ExprVisitor):

    def visitProg(self, ctx):
        return self.visit(ctx.e())

    # e : e '+' t # SumaExpr
    def visitSumaExpr(self, ctx):
        izquierda = self.visit(ctx.e())
        derecha = self.visit(ctx.t())
        return izquierda + derecha

    # e : e '-' t # RestaExpr
    def visitRestaExpr(self, ctx):
        izquierda = self.visit(ctx.e())
        derecha = self.visit(ctx.t())
        return izquierda - derecha

    # e : t # TerminoComoExpr
    def visitTerminoComoExpr(self, ctx):
        return self.visit(ctx.t())

    # t : t f # MultiplicacionImplicita
    def visitMultiplicacionImplicita(self, ctx):
        izquierda = self.visit(ctx.t())
        derecha = self.visit(ctx.f())
        return izquierda * derecha

    # t : t '*' f # MultiplicacionExplicita
    def visitMultiplicacionExplicita(self, ctx):
        izquierda = self.visit(ctx.t())
        derecha = self.visit(ctx.f())
        return izquierda * derecha

    # t : f # FactorComoTermino
    def visitFactorComoTermino(self, ctx):
        return self.visit(ctx.f())

    # f : ID # FactorID
    def visitFactorID(self, ctx):
        # No hay tabla de símbolos definida aún; se reporta el error
        nombre = ctx.ID().getText()
        raise ValueError(f"Identificador '{nombre}' no tiene valor asignado (no hay tabla de símbolos)")

    # f : NUM # FactorNum
    def visitFactorNum(self, ctx):
        texto = ctx.NUM().getText()
        if '.' in texto:
            return float(texto)
        return int(texto)

    # f : '(' e ')' # FactorParentesis
    def visitFactorParentesis(self, ctx):
        return self.visit(ctx.e())
```

Figura 7: Visitor para evaluacion numerica

```
# main.py

import sys
from antlr4 import InputStream, FileStream, CommonTokenStream
from ExprLexer import ExprLexer
from ExprParser import ExprParser
from ExprEvalVisitor import ExprEvalVisitor

def procesar(entrada_stream, etiqueta):
    lexer = ExprLexer(entrada_stream)
    tokens = CommonTokenStream(lexer)
    parser = ExprParser(tokens)
    arbol = parser.prog()

    print(f"{'etiqueta}'")
    print(f"  Árbol : {arbol.toStringTree(recog=parser)}")

    try:
        visitor = ExprEvalVisitor()
        resultado = visitor.visit(arbol)
        print(f"  Valor : {resultado}")
    except ValueError as e:
        print(f"  Valor : ERROR - {e}")

def main():
    if len(sys.argv) < 2:
        texto = input(" ")
        entrada = InputStream(texto)
        procesar(entrada, texto)
    elif sys.argv[1] == '-f':
        if len(sys.argv) < 3:
            print("Error: falta la ruta del archivo tras -f")
            sys.exit(1)
        ruta = sys.argv[2]
        entrada = FileStream(ruta, encoding='utf-8')
        procesar(entrada, f"archivo:{ruta}")
    else:
        texto = sys.argv[1]
        entrada = InputStream(texto)
        procesar(entrada, texto)

if __name__ == '__main__':
    main()
```

Figura 8: actualizar main para mostrar evaluación

2.2. Pruebas y análisis

```
santiago-patino@vivobook:~/Escritorio/analisis-sintactico/punto1$ echo "Prueba gramatica + evaluacion"
Prueba gramatica + evaluacion
santiago-patino@vivobook:~/Escritorio/analisis-sintactico/punto1$ python3 main.py "2 + 3 * 4"
Árbol : (prog (e (e (t (f 2))) + (t (t (f 3)) * (f 4))) <EOF>)
Valor : 14
santiago-patino@vivobook:~/Escritorio/analisis-sintactico/punto1$ python3 main.py "2 + 3(4 - 5)"
Árbol : (prog (e (e (t (f 2))) + (t (t (f 3)) (f { (e (e (t (f 4))) - (t (f 5))) }))) <EOF>)
Valor : -1
santiago-patino@vivobook:~/Escritorio/analisis-sintactico/punto1$ python3 main.py "2 + 3 - 4"
Árbol : (prog (e (e (e (t (f 2))) + (t (f 3))) - (t (f 4))) <EOF>)
Valor : 1
santiago-patino@vivobook:~/Escritorio/analisis-sintactico/punto1$
```

Figura 9: prueba gramática + evaluación

Tabla 1: Resultados del punto 1

Expresión	Agrupación	Obtenido	Esperado
$2 + 3 * 4$	$2 + (3 \cdot 4)$	14	14
$2 + 3(4 - 5)$	$2 + (3 \cdot (4 - 5))$	-1	-1
$2 + 3 - 4$	$(2 + 3) - 4$	1	1
$2 + 3 * 4 - 1$	$(2 + (3 \cdot 4)) - 1$	13	13

Los cuatro resultados coinciden con la aritmética. En $2+3*4$ el producto aparece dentro del hijo derecho de la suma, es decir más abajo en el árbol, y como el Visitor evalúa de abajo hacia arriba se resuelve primero: la precedencia no está escrita en el código, es consecuencia de la jerarquía $E-T-F$. En $2+3-4$ la suma queda anidada dentro de la resta, de donde $(2 + 3) - 4 = 1$ y no $2 + (3 - 4) = 3$.

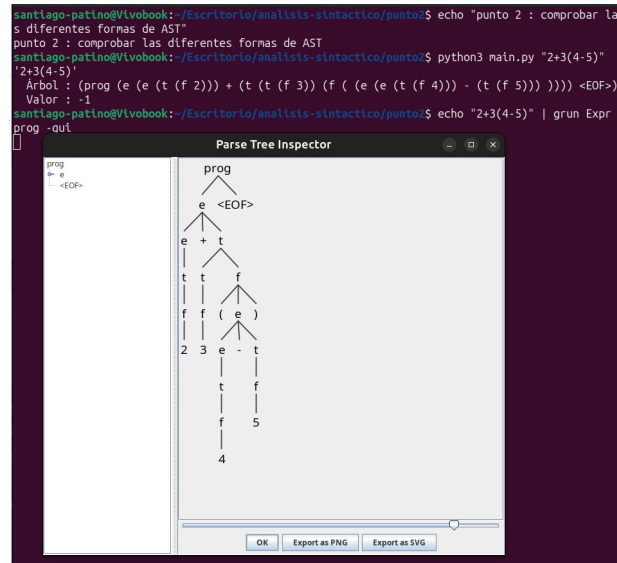
Un efecto colateral que conviene declarar: la alternativa $\mathbf{t} : \mathbf{t} \mathbf{f}$ hace que dos factores yuxtapuestos se lean siempre como producto, de modo que una entrada como $2 \ 3$ no se rechaza, sino que se evalúa como 6. Es una decisión de diseño, no un error del analizador.

3. Punto 2. Formas del árbol (diapositiva 12)

ANTLR4 no construye un AST: construye siempre el **parse tree** concreto —con todos los nodos intermedios y todos los terminales, incluidos los paréntesis— y ofrece de él varias representaciones equivalentes. El AST se obtiene después, recorriendo ese árbol con un Visitor.

Tabla 2: Parse tree frente a AST

Parse tree (concreto)	AST (abstracto)
Un nodo por cada regla aplicada, incluidas las cadenas unitarias ($E \rightarrow T \rightarrow F \rightarrow num$).	Solo los nodos con contenido semántico; las cadenas unitarias desaparecen.
Conserva paréntesis y el operador como hoja.	Sin paréntesis; el operador pasa a ser el nodo padre.



3.1. Las dos formas comprobadas

Forma textual (tipo LISP). `toStringTree(recog=parser)` devuelve el árbol entre paréntesis; es la forma cómoda para documentar y comparar.

Forma gráfica. `grun` con la opción `-gui` abre el *Parse Tree Inspector*, que además exporta a PNG o SVG. Requiere haber generado también el destino Java, pero la gramática es la misma, así que el árbol mostrado es el mismo que produce el analizador de Python.

```
python3 main.py "2+3(4-5)"
echo "2+3(4-5)" | grun Expr prog -gui
```

Listing 2: Las dos representaciones del mismo árbol

Caso A: 2+3(4-5)

```
(prog (e (e (t (f 2))) + (t (t (f 3)) (f ( (e (e (t (f 4))) - (t (f 5))) ))) ) <EOF>)
Valor : -1
```

```
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto2$ echo "punto 2 : comprobar las diferentes formas de AST"
punto 2 : comprobar las diferentes formas de AST
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto2$ python3 main.py "2+3(4-5)"
Árbol : (prog (e (e (t (f 2))) + (t (t (f 3)) (f ( (e (t (f 4))) - (t (f 5))) ))) <EOF>)
Valor : -1
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto2$ echo "2+3(4-5)" | grun Expr prog -out
```

Caso B: 2+3*4-1

```
(prog (e (e (e (t (f 2))) + (t (t (f 3)) * (f 4))) - (t (f 1))) <EOF>)
Valor : 13
```

```
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto2$ echo "punto 2 : comprobar las diferentes formas de AST"
punto 2 : comprobar las diferentes formas de AST
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto2$ python3 main.py "2+3*4-1"
Árbol : (prog (e (e (e (t (f 2))) + (t (t (f 3)) * (f 4))) - (t (f 1))) <EOF>)
Valor : 13
santiago-patino@Vivobook:~/Escritorio/analisis-sintactico/punto2$ echo "2+3*4-1" | grun Expr prog -out
```

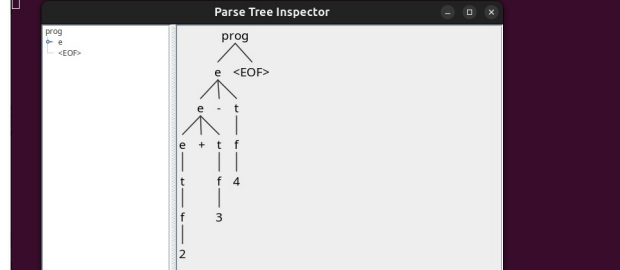
Caso C: 2+3-4

```
(prog (e (e (e (t (f 2))) + (t (f 3))) - (t (f 4))) <EOF>)
Valor : 1
```

```

santiago-patino@Vivobook: ~/Escritorio/analisis-sintactico/punto2$ echo "punto 2 : comprobar la
s diferentes formas de AST"
punto 2 : comprobar las diferentes formas de AST
santiago-patino@Vivobook: ~/Escritorio/analisis-sintactico/punto2$ python3 main.py "2+3-4"
'2+3-4'
Árbol : (prog (e (e (t (f 2))) + (t (f 3))) - (t (f 4))) <EOF>)
Valor : 1
santiago-patino@Vivobook: ~/Escritorio/analisis-sintactico/punto2$
santiago-patino@Vivobook: ~/Escritorio/analisis-sintactico/punto2$ echo "2+3-4" | grun Expr pro
g -gui

```



3.2. Análisis

Las dos formas son la misma estructura: cada paréntesis de la notación LISP corresponde a un nivel de la ventana gráfica. En el caso A la forma gráfica muestra lo que la textual deja implícito —los paréntesis aparecen como **hojas** del nodo **f**—, y son justamente esas hojas las que un AST eliminaría, porque la agrupación ya está dada por la forma del árbol.

La distancia entre ambos árboles se mide en nodos vacíos: para $2+3-4$, el parse tree usa la cadena $e \rightarrow t \rightarrow f \rightarrow \text{NUM}$ para llegar a cada número, mientras que el AST equivalente tendría cinco nodos en total. El Visitor es precisamente el paso de uno al otro: al escribir `visitTerminoComoExpr` como `return self.visit(ctx.t())` se colapsa una cadena unitaria, y al ignorar los paréntesis en `visitFactorParentesis` se descartan terminales sin contenido semántico.

4. Punto 3. Gramática ambigua (diapositiva 15)

4.1. Planteamiento

La gramática $E \rightarrow E + E \mid E * E \mid \text{num}$ no tiene niveles: suma y producto viven en la misma regla. Nada indica cuál debe quedar más abajo en el árbol, de modo que $2 + 3 * 4$ admite dos derivaciones: $2 + (3 * 4) = 14$ y $(2 + 3) * 4 = 20$. Misma cadena, dos árboles, dos significados.

```

1 grammar Ambigua;
2
3 prog : e EOF ;
4
5 e : e '+' e # SumaAmbigua
6   | e '*' e # MultAmbigua
7   | NUM     # NumAmbigua
8   ;
9
10 NUM : [0-9]+ ('.' [0-9]+)? ;
11 WS  : [ \t\r\n]+ -> skip ;

```

Listing 3: Ambigua.g4

```

GNU nano 7.2
grammar Ambigua;

prog
: e EOF
;

e
: e '+' e # SumaAmbigua
| e '*' e # MultAmbigua
| NUM    # NumAmbigua
;

NUM : [0-9]+ ('.' [0-9]+)? ;
WS  : [ \t\r\n]+ -> skip ;

```

4.2. Generación y ejecución

Se generó el analizador en los dos destinos: Python para evaluar, y Java para poder inspeccionar el árbol con `grun -gui`.

```

antlr4 -Dlanguage=Python3 -visitor Ambigua.g4
antlr4 Ambigua.g4
javac Ambigua*.java
echo "2 + 3 * 4" | grun Ambigua prog -gui

```

Listing 4: Generación en Python y en Java

```

santiago-patino@Vivobook: ~/Escritorio/analisis-sintactico/punto3$ echo "Generar el parser en Python"
Generar el parser en Python
santiago-patino@Vivobook: ~/Escritorio/analisis-sintactico/punto3$ antlr4 -Dlanguage=Python3 -visitor Ambigua.g4
santiago-patino@Vivobook: ~/Escritorio/analisis-sintactico/punto3$ antlr4 Ambigua.g4
santiago-patino@Vivobook: ~/Escritorio/analisis-sintactico/punto3$

```

Figura 10: generar el parser

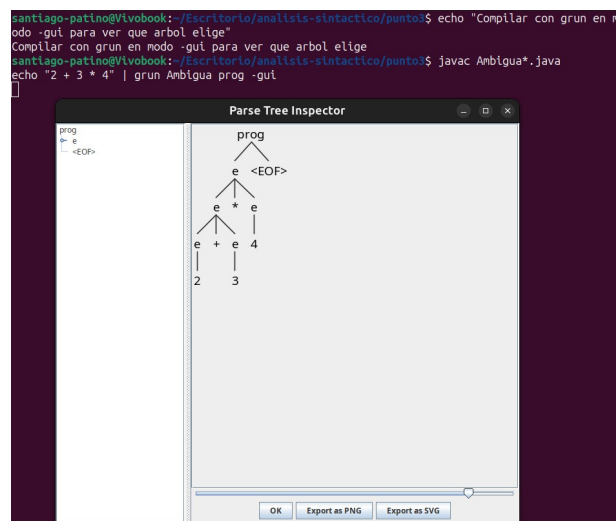


Figura 11: Compilar y correr con `grun` en modo `-gui` para ver qué árbol elige

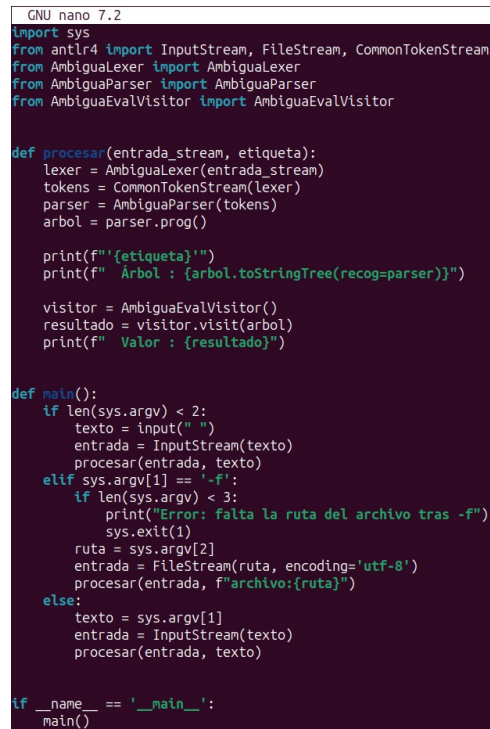
El evaluador es análogo al del punto 1, pero como ambos operandos son `e`, se distinguen por índice: `ctx.e(0)` y `ctx.e(1)`.

```

1 class AmbiguaEvalVisitor(AmbiguaVisitor):
2
3     def visitProg(self, ctx):
4         return self.visit(ctx.e())
5
6     def visitSumaAmbigua(self, ctx):
7         izquierda = self.visit(ctx.e(0))
8         derecha = self.visit(ctx.e(1))
9         return izquierda + derecha
10
11     def visitMultAmbigua(self, ctx):
12         izquierda = self.visit(ctx.e(0))
13         derecha = self.visit(ctx.e(1))
14         return izquierda * derecha
15
16     def visitNumAmbigua(self, ctx):
17         texto = ctx.NUM().getText()
18         return float(texto) if '.' in texto else int(texto)

```

Listing 5: AmbiguaEvalVisitor.py



```

GNU nano 7.2
import sys
from antlr4 import InputStream, FileStream, CommonTokenStream
from AmbiguaLexer import AmbiguaLexer
from AmbiguaParser import AmbiguaParser
from AmbiguaEvalVisitor import AmbiguaEvalVisitor

def procesar(entrada_stream, etiqueta):
    lexer = AmbiguaLexer(entrada_stream)
    tokens = CommonTokenStream(lexer)
    parser = AmbiguaParser(tokens)
    arbol = parser.prog()

    print(f'"{etiqueta}"')
    print(f"  Árbol : {arbol.toStringTree(recog=parser)}")

    visitor = AmbiguaEvalVisitor()
    resultado = visitor.visit(arbol)
    print(f"  Valor : {resultado}")

def main():
    if len(sys.argv) < 2:
        texto = input(" ")
        entrada = InputStream(texto)
        procesar(entrada, texto)
    elif sys.argv[1] == '-f':
        if len(sys.argv) < 3:
            print("Error: falta la ruta del archivo tras -f")
            sys.exit(1)
        ruta = sys.argv[2]
        entrada = FileStream(ruta, encoding='utf-8')
        procesar(entrada, f"archivo:{ruta}")
    else:
        texto = sys.argv[1]
        entrada = InputStream(texto)
        procesar(entrada, texto)

if __name__ == '__main__':
    main()

```

Figura 12: main

```

GNU nano 7.2
from AmbiguaVisitor import AmbiguaVisitor

class AmbiguaEvalVisitor(AmbiguaVisitor):

    def visitProg(self, ctx):
        return self.visit(ctx.e())

    # e : e '+' e # SumaAmbigua
    def visitSumaAmbigua(self, ctx):
        izquierda = self.visit(ctx.e(0))
        derecha = self.visit(ctx.e(1))
        return izquierda + derecha

    # e : e '*' e # MultAmbigua
    def visitMultAmbigua(self, ctx):
        izquierda = self.visit(ctx.e(0))
        derecha = self.visit(ctx.e(1))
        return izquierda * derecha

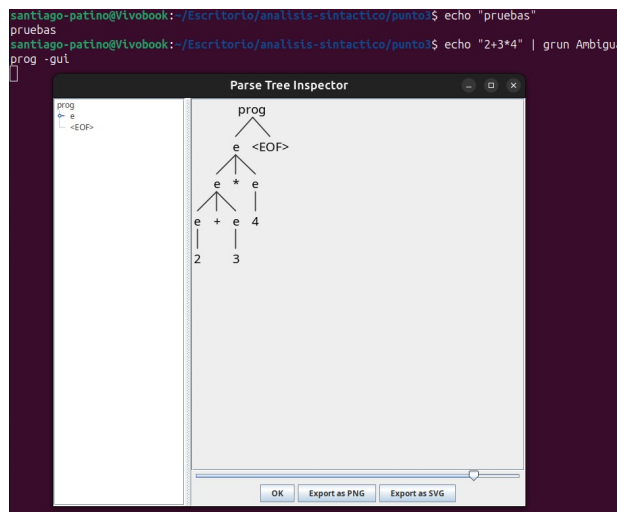
    # e : NUM # NumAmbigua
    def visitNumAmbigua(self, ctx):
        texto = ctx.NUM().getText()
        if '.' in texto:
            return float(texto)
        return int(texto)

```

Figura 13: Visitor para evaluación numérica

4.3. Resultado: qué árbol elige ANTLR

ANTLR no rechaza la gramática ambigua ni reporta error: la **resuelve** y sigue adelante. Para reglas recursivas por la izquierda, su criterio es el **orden de las alternativas**: la que aparece primero recibe mayor precedencia. Como `e '+' e` está escrita antes que `e '*' e`, la suma se agrupa primero y el árbol elegido para $2 + 3 * 4$ es $(2 + 3) \cdot 4$, que aritméticamente es incorrecto.



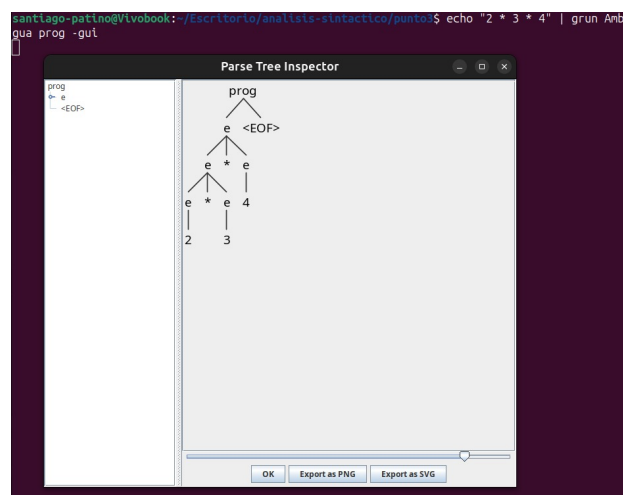
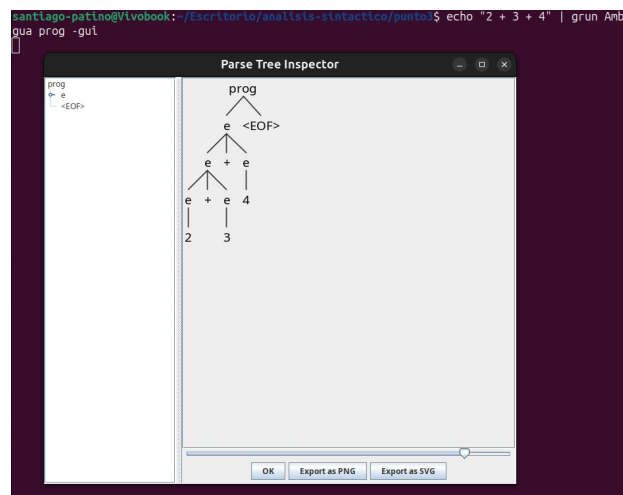
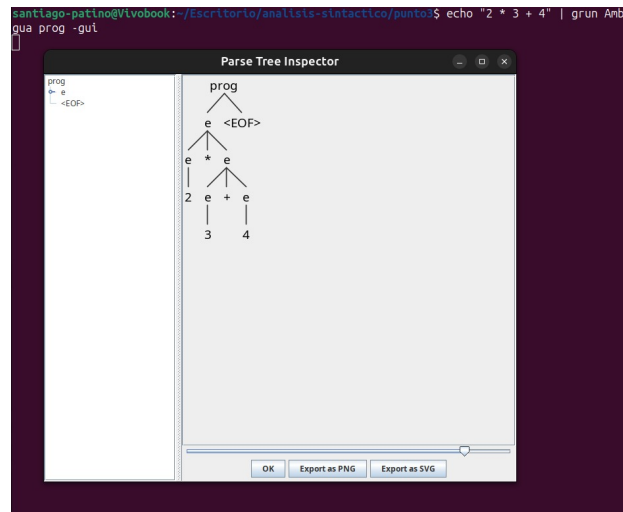


Tabla 3: Árbol elegido según el orden de las alternativas, para $2 + 3 * 4$

Orden en el .g4	Árbol elegido	Valor
'+' antes que '*' (versión entregada)	$(2 + 3) \cdot 4$	20
'*' antes que '+'	$2 + (3 \cdot 4)$	14

Que la misma cadena produzca dos árboles distintos con solo intercambiar el orden de dos líneas es la prueba de que ambos árboles son derivaciones válidas de la gramática y, por tanto, de que la gramática es ambigua. A ello se suma que ANTLR permite detectarlo de forma explícita con el `DiagnosticErrorListener` y el modo `LL_EXACT_AMBIG_DETECTION`, o con `grun Ambigua prog -diagnostics`, que reporta `reportAmbiguity`.

4.4. Contraste con el punto 1

	Punto 1 (E, T, F)	Punto 3 ($E \rightarrow E + E \mid E * E$)
Árboles para $2 + 3 * 4$	Uno	Dos
Origen de la precedencia	La jerarquía de no terminales	El orden de las líneas en el .g4
Riesgo	Ninguno	Reordenar reglas cambia el significado sin producir ningún error

Esa última fila es la moraleja del punto: una gramática ambigua no falla ruidosamente. Compila, ejecuta y devuelve un número; simplemente puede devolver el número equivocado, y el error solo aparece cuando alguien compara el resultado con la aritmética.

5. Conclusiones

1. La gramática estratificada en E , T y F codifica la precedencia y la asociatividad en su propia forma: las cuatro expresiones evaluadas dieron el resultado aritmético correcto sin que el evaluador contenga ninguna regla de precedencia.
2. ANTLR construye siempre un parse tree concreto y ofrece de él representaciones equivalentes —textual y gráfica—. El AST no es una estructura que la herramienta genere, sino el resultado de recorrer el árbol concreto descartando los nodos sin contenido semántico; el Visitor es el mecanismo que realiza ese paso.
3. Una gramática ambigua no produce error de compilación: ANTLR resuelve el conflicto con el orden de las alternativas y entrega uno de los árboles posibles. La prueba de $2 + 3 * 4$ lo confirma, porque el árbol elegido agrupa primero la suma.
4. En consecuencia, la ambigüedad debe comprobarse de forma deliberada, y en el diseño de un lenguaje la solución correcta es estratificar la gramática como en el punto 1 en vez de confiar en la desambiguación automática de la herramienta.

Referencias

- Parr, T. (2013). *The Definitive ANTLR 4 Reference* (2ª ed.). The Pragmatic Bookshelf.
- Aho, A., Lam, M., Sethi, R. y Ullman, J. (2008). *Compiladores: principios, técnicas y herramientas* (2ª ed.). Pearson.
- Documentación oficial de ANTLR 4. <https://github.com/antlr/antlr4/blob/master/doc/index.md>